Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 6

Stack

Stack

- What is a stack?
  - Objects (say books) are kept one on top of the other
  - An item can be added only to the top of the stack
  - Can remove only the top-most item (the most recently added)
  - Last-In First-Out (LIFO)
- Why are stacks useful in programs?
  - To support "undo", a stack can used to store actions so far
  - Algorithms for parsing a string according to syntax rules ("grammar") use a stack
  - Systematic searches often use a stack too
  - Memory allocated for function calls naturally use a stack (last function invoked is the first to return)
  - …

A Quick Stack Implementation

```
struct charStack {
    vector<char> stk;
    bool pop(char& x) { // if stack is empty, return false
        if (stk.empty()) return false;
        x = stk.back(); stk.pop_back(); // back is top
        return true;
    }
    void push(char x) {
        stk.push_back(x); // back is top
    }
    void clear() { stk.clear(); }
    bool empty() { return stk.empty(); }
};
```

Example: Palindrome using a Stack

- Check if an input string is a palindrome

```
int main() {
  charStack st;
  char x, y;
  int n; std::cin >> n; // read the length of the string first
  for (int i=0; i<n/2; i++) { std::cin >> x; st.push(x); }
  if (n%2) std::cin >> x; // if n odd, ignore the middle character
  for (int i=0; i<n/2; i++) { // compare stored characters with rest
    std::cin >> x;
    st.pop(y); // guaranteed to succeed since we pushed n/2 items
    if(x != y) { std::cout << "Not a palindrome!" << std::endl; return 1; }
  }
  std::cout << "Palindrome!" << std::endl;
}
```

assuming no input error

Example: Undo Stack

```
bool docommand(char c, charStack& history); // push command into history
bool undo(charStack& history); // pop one command from history & reverse it
int main() {
  turtleSim();  // our commands are for the turtle (from CS101)
  cout << "Enter f/r/l for forward/right/left, u to undo, q to quit: ";
  charStack history; history.clear();
  char input;
  for(cin >> input; !cin.fail(); cin >> input) {
    if(input == 'q')
      break;
    else if (input == 'u')
      if(!undo(history)) // try undoing
        cerr << "Undo stack empty!\n";
    else if (!docommand(input,history)) // try doing
      cerr << "Ignoring unrecognized command" << endl;
  }
}
```

Example: Undo Stack

Demo

```
bool docommand(char c, charStack& history) {
  if (c == 'f' || c == 'r' || c == 'l') {
    history.push(c);
    if (c == 'f') forward(10);
    else if (c == 'r') right(90);
    else /* (c == 'l') */ left(90);
    return true;
  }
  return false;
}
```

```
bool undo(charStack& history) {
  char c;
  if(!history.pop(c)) {
    return false;
  } else {
    if (c == 'f') forward(-10);
    else if (c == 'r') right(-90);
    else /* (c == 'l') */ left(-90);
    return true;
  }
}
```

Vector Problems

- A vector provides more features than a stack
  - In particular, random access
  - But comes at a cost!
    - To keep data contiguous, reallocates memory and moves data, if vector grows beyond the reserved size.
      - Moving data from one vector to another costs O(n) time in the worst-case
- A stack implementation doesn't need to suffer this cost!

Lists for Stacks

- If we had an a priori bound on the stack size, an array/vector is an excellent fit for implementing a stack
  - It supports O(1) adding and deleting items at the end (stack's top), with concrete speeds much faster than a linked list
- The only problem is that the stack may outgrow the array
- Idea: If a fixed-size stack fills up, add a second fixed-size stack on top of it
  - The first stack should be accessible from the one on top (if the top one is fully popped): top stack should point to the one below it
  - A linked list of fixed-size stacks!

A Fixed-Capacity Stack

```
struct stackBlock {
   const int capacity = 512; // fixed capacity
   char stk[capacity]; // will use an array instead of std::vector
   int top = -1; // keep track of last element's index. -1 for empty.
   bool pop(char& x) { // if stack is empty, return false
        if (top < 0) return false;
        x = stk[top--]; // decrement top after retrieving into x
        return true;
    }
    bool try_push(char x) { // if stack already full, return false
        if(top == capacity-1) return false;
        stk[++top] = x; return true; // increment top before storing x
    }
    void clear() { top = -1; }
    bool empty() { return top==-1; }
};
```

A List of Bounded Stacks

```
struct stackNode { stackBlock block; stackNode* below = nullptr; }
struct stackList { // a list of stackNodes
  stackNode* head = nullptr; // top most stackNode
  bool empty() { return head==nullptr; } // stack empty iff list empty
  void push(char x) {
    if(head==nullptr) head = new stackNode; // if adding to empty list
    if(!head->block.try_push(x)) { // try pushing to top stack
      stackNode* tmp = new stackNode; tmp->below = head; head = tmp;
      head->block.try_push(x); // push again after adding a new node
    }
  }
  bool pop(char& x) {
    if (empty()) return false;
    head->block.pop(x); // if not empty, pop from top stack
    if(head->block.empty()) { // if it is empty after popping, delete it
      stackNode* tmp = head->below; delete head; head = tmp;
    }
    return true;
  }
};
```

Exercise: An Issue to Fix

- In this stackList, a series of strictly alternating pushes and pops at the beginning (or at the beginning of a new stackBlock), each push/pop will result in adding/deleting a stockBlock
  - Still O(1) worst-case, but larger constants
- An idea: Add a new block on top when the top block is full (like before); but don't delete the top block when it turns empty -- delete only when the block below becomes empty
  - Allow for the possibility that head is not where the top is
- Exercise

Forgetful Stack

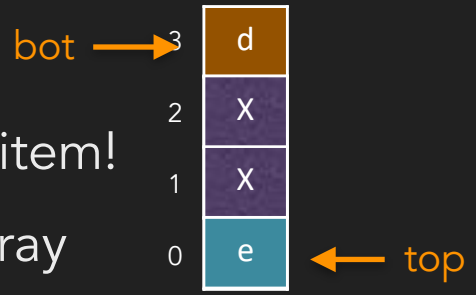
- An unlimited stack may not be suitable for many applications
- E.g., undo stack in an image editor
  - The information required to undo an action could be large (e.g., a snapshot of the image before a lossy blur operation)
  - Storing the undo information for all the commands in an editing session can grow to Giga Bytes of memory.
  - Better to have an a priori bound on the size of the undo stack
- Idea: When pushing a new item into a full stack, we can just forget the oldest item

Forgetful Stack

- How can you efficiently implement a forgetful stack
- Since fixed size, an array suffices: No need for a linked list or std::vector
- But what should we do once the array fills up?
  - Naively: Move all the items in the array to the left (letting the left-most item disappear) and make room for the new item at the right end
    - Problem: Push becomes $\Theta(n)$
  - We shouldn't move the items already in the stack, except possibly the one at the bottom (which is getting forgotten)
  - So we must overwrite the bottom element with the new item!

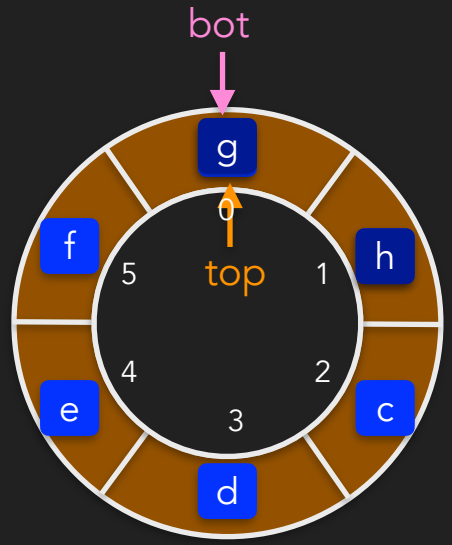
Forgetful Stack

- How can you efficiently implement a forgetful stack
- Since fixed size, an array suffices: No need for a linked list or std::vector
- But what should we do once the array fills up?
  - We must overwrite the bottom element with the new item!
- Top position moves around, modulo N, the size of the array
- Bottom position also moves mod N (only when pushed up by top)
- Stack size can be any of N+1 values (0 through N), but the gap between top and bot can be only one of N values (0 through N-1)
  - Will use a flag to distinguish between two cases (zero or one item)



Forgetful Stack

```
struct stackRing {
  const int N = 512; // capacity
  char array[N];
  int top = 0, bot = 0;
  bool is_empty = true; // for us top==bot can mean 0 or 1 item
  void inc(int& i) { i = (i==N-1 ? 0:i+1); } // increment mod N
  void dec(int& i) { i = (i==0 ? N-1:i-1); } // decrement mod N
  void push(char x) {
    if(is_empty) is_empty = false; // top and bot stay put
    else { inc(top); if(top==bot) inc(bot); }
    array[top] = x;
  }
  bool pop(char& x) {
    if(is_empty) return false;
    x = array[top];
    if(top==bot) is_empty = true; // had one item: don't move top
    else dec(top);
    return true;
  }
};
```



Summary

- A stack provides only push and pop operations: sufficient for many interesting applications
- No random access and hence no need to maintain items contiguously
  - Implementation using a vector is not a good fit
- We saw three implementations with O(1) worst-case push/pop
  - stackBlock: A stack with a pre-determined capacity, using an array
  - stackList: An (uncapped) stack using a list of stackBlocks
  - stackRing: A stack with a pre-determined capacity, that forgets oldest items, using an array